PixelVault coordination handoff for splitting responsibilities out of MainWindow

How to use this PDF: keep Cursor on UI-host and MainWindow surface work, keep Codex on new services/models/tests, and let only one side own the final integration pass in `PixelVault.Native.cs` or `ImportWorkflow.cs`.

Service Ownership And Parallel Work Map

Purpose

This is the practical coordination map for continuing the MainWindow split without stepping on each other.

Use it to answer three questions:

1. What already counts as a real service or host?
2. What still lives in `MainWindow` or partials?
3. Which slices are safe to do in parallel between Cursor and Codex?

This document is based on the current code in `C:\Codex\src\PixelVault.Native` and complements:

- `C:\Codex\docs\MAINWINDOW_EXTRACTION_ROADMAP.md`
- `C:\Codex\docs\pixelvault_service_split_plan.txt`

Current Service Map

Area	Current type(s)	File(s)	What it owns now	What still lives outside it	Status
Cover art and external lookups	`ICoverService`, `CoverService`	`src/PixelVault.Native/Services/Covers/CoverService.cs`	Steam app lookups, SteamGridDB lookups, cover downloads, custom cover file paths, cache naming, deleting cached covers	Library toolbar/button wiring, image rendering, library refresh flow, some title/console normalization delegated from `MainWindow`	Extracted service, still depends on `MainWindow` helpers
Metadata read/write	`IMetadataService`, `MetadataService`	`src/PixelVault.Native/Services/Metadata/MetadataService.cs`	ExifTool arg building, direct metadata reads, batch metadata reads, write batching, sidecar handling, timestamp restoration	Metadata policy helpers (`BuildMetadataTagSet`, comment/tag cleanup), metadata editor workflow orchestration, UI progress + status text	Extracted service, behavior still partly defined by `MainWindow` delegates
Index persistence and filename rule storage	`IIndexPersistenceService`, `IndexPersistenceService`	`src/PixelVault.Native/Services/Indexing/IndexPersistenceService.cs`	SQLite-backed game/photo index persistence, filename convention persistence, filename convention samples, alias persistence/migration	Scan/rebuild orchestration, folder grouping, index editor UI actions, some normalization/alias rewrite behavior still delegated from `MainWindow`	Extracted persistence layer, not yet a full library/index domain service
Filename parsing	`IFilenameParserService`, `FilenameParserService`	`src/PixelVault.Native/Services/FilenameParsing/FilenameParserService.cs`	Built-in + custom filename rule loading, parsing filenames into a `FilenameParseResult`, generic capture-date fallback, Steam rename hint logic	Import workflow consequences of parse results, manual intake routing, rename/move path propagation, UI-facing display decisions	Extracted parser, still consumed directly from `MainWindow` /import partials
Filename rules editor state	`IFilenameRulesService`, `FilenameRulesService`	`src/PixelVault.Native/Services/FilenameRules/FilenameRulesService.cs`	Load/save rules, create rules, normalize/validate save payload, dismiss samples	Editor window layout, editor control wiring, status/log handling	Good separation already

Current UI Host Map

These are not "services", but they are important extraction seams because they remove UI surface area from `PixelVault.Native.cs`.

Surface	Current type(s)	File(s)	What it owns now	Status
Changelog window	`ChangelogWindow`	`src/PixelVault.Native/UI/ChangelogWindow.cs`	Changelog dialog UI	Extracted
Progress shell	`WorkflowProgressWindow`, `WorkflowProgressView`	`src/PixelVault.Native/UI/Progress/WorkflowProgressWindow.cs`	Shared progress window layout, log box, progress bar, footer action button	Extracted
Intake preview	`IntakePreviewWindow`, `IntakePreviewServices`	`src/PixelVault.Native/UI/Intake/IntakePreviewWindow.cs`	Upload queue preview modal UI	Extracted host
Metadata review	`MetadataReviewWindow`, `MetadataReviewServices`	`src/PixelVault.Native/UI/Intake/MetadataReviewWindow.cs`	Pre-import metadata review UI	Extracted host
Filename rules editor window	`FilenameConventionEditorWindow`, `FilenameConventionEditorServices`	`src/PixelVault.Native/UI/Editors/FilenameConventionEditorWindow.cs`	Filename rules modal UI	Extracted host
Library virtualization primitives	partial `MainWindow` helpers	`src/PixelVault.Native/UI/LibraryVirtualization.cs`	Virtualized rows/scroll host behavior	Extracted partial, not yet a standalone host

Areas Still Centered On MainWindow

These are the big remaining ownership zones even though some helpers are already split out.

Area	Main files	Why it is still "MainWindow-owned"
App bootstrapping + composition	`src/PixelVault.Native/PixelVault.Native.cs`	`MainWindow` constructs services directly and passes many behavior delegates into them
Settings/config	`src/PixelVault.Native/PixelVault.Native.cs`	Paths, settings load/save, defaults, and config UI still live in `MainWindow`
Import orchestration	`src/PixelVault.Native/Import/ImportWorkflow.cs`	Rename, delete, metadata, move, sort, undo manifest, and workflow progress still live as a `MainWindow` partial
Library browser shell	`src/PixelVault.Native/PixelVault.Native.cs`, `src/PixelVault.Native/UI/LibraryVirtualization.cs`	Folder tiles, detail pane, toolbar actions, search, selection, refresh, and library scan workflow are still heavily window-owned
Library scanning/grouping	`src/PixelVault.Native/Indexing/LibraryFolderIndexing.cs`, `src/PixelVault.Native/Indexing/LibraryMetadataIndexing.cs`, `src/PixelVault.Native/Indexing/GameIndexCore.cs`, `src/PixelVault.Native/Indexing/GameIndexFolderAlignment.cs`	Logic is split across partials/helpers, but there is not yet a dedicated `LibraryScanner` / `LibraryService` boundary
Metadata editing flows	`src/PixelVault.Native/Metadata/LibraryMetadataEditing.cs`, `src/PixelVault.Native/Metadata/MetadataHelpers.cs`	The ExifTool service exists, but metadata policy and edit orchestration still lean on window state
Logging, file system, process execution	`src/PixelVault.Native/PixelVault.Native.cs`, `src/PixelVault.Native/MediaTools/MediaToolHelpers.cs`	Cross-cutting helpers still come from `MainWindow` and direct `System.IO` / process calls

---

Missing Or Incomplete Service Candidates

These are the best next service seams, ordered by usefulness and risk.

Candidate	Why it matters	Suggested file/home	Risk	Notes
`ISettingsService` / `SettingsService`	Reduces path/config noise and secrets handling in `MainWindow`	`src/PixelVault.Native/Services/Config/SettingsService.cs`	Low	Best next true service extraction
`IImportService` / `ImportService`	Converts `ImportWorkflow.cs` from a `MainWindow` partial into a domain service	`src/PixelVault.Native/Services/Import/ImportService.cs`	Medium	Good boundary because the workflow is already clustered in one file
`ILibraryScanner` / `LibraryScanner`	Pulls scan/rebuild/group logic out of library UI code	`src/PixelVault.Native/Services/Library/LibraryScanner.cs`	Medium-High	Biggest leverage after settings/import, but touches many helpers
`ILibraryBrowserHost` / `LibraryBrowserHost`	Makes the main library window a host instead of one giant construction method	`src/PixelVault.Native/UI/Library/LibraryBrowserHost.cs`	High	Strong UI extraction target, but avoid pairing it with service extraction in the same edit pass
`ILogService`	Centralizes log writes and operational messages	`src/PixelVault.Native/Services/Logging/LogService.cs`	Low-Medium	Helpful, but not the first slice unless logging is actively hurting productivity
`IFileSystemService`	Reduces direct `System.IO` calls and helps testability	`src/PixelVault.Native/Services/IO/FileSystemService.cs`	Medium	Useful support seam for import/library later

---

Recommended Parallel Lanes

The most important rule: do not have both tools editing `PixelVault.Native.cs` or `ImportWorkflow.cs` at the same time unless one side is only reading.

Lane A: UI Host / MainWindow Surface Work

Best owner: Cursor

Write scope:

- `src/PixelVault.Native/PixelVault.Native.cs`
- `src/PixelVault.Native/UI/**/*`
- small related docs if needed

Good tasks:

- continue extracting window-building blocks out of `PixelVault.Native.cs`

- move library browser sections into a dedicated host class
- reduce WPF layout/event wiring in the main file
- keep `MainWindow` focused on composition + top-level navigation

Avoid pairing with:

- Codex changing the same partial class or the same UI host files

Lane B: Service / Model / Test Work

Best owner: Codex

Write scope:

- `src/PixelVault.Native/Services/**/*`
- `src/PixelVault.Native/Models/**/*`
- `tests/PixelVault.Native.Tests/**/*`
- service-oriented docs

Good tasks:

- introduce a new service interface + implementation
- move pure business logic out of partials into new service classes
- add unit tests for extracted service behavior
- create DTO/result models so UI does not pass raw dictionaries everywhere

Avoid pairing with:

- Cursor rewriting the same method bodies in `ImportWorkflow.cs` or `PixelVault.Native.cs` during the same pass

Lane C: Integration Pass

Best owner: one person at a time

Write scope:

- `src/PixelVault.Native/PixelVault.Native.cs`
- whichever partial currently wires the extracted logic

Good tasks:

- replace direct method calls with service calls
- pass dependencies into a new host/service
- remove dead helpers after extraction

Rule:

- only one tool should own the integration commit

Safe Parallel Slices Right Now

These are the slices that are easiest to split between Cursor and Codex without merge pain.

Cursor focus	Codex focus	Why this pairing is safe
Extract more Library UI out of `PixelVault.Native.cs` into `UI/Library/*`	Build `ISettingsService` and tests in new `Services/Config/*` files	Different write sets until final wiring
Continue window/host extraction under `UI/Intake/*` or `UI/Editors/*`	Build `IImportService` skeleton, result models, and unit tests in new `Services/Import/*` files	Service work can happen first without touching the current WPF host
Clean up toolbar/layout/event-wiring sections in `PixelVault.Native.cs`	Build `ILibraryScanner` models and persistence-facing facade in `Services/Library/*`	Codex can prepare domain objects while Cursor works on the surface
Refine `LibraryBrowserHost` or equivalent UI extraction	Harden `MetadataService` / `IndexPersistenceService` tests and move more pure helpers behind them	Test/service hardening does not need the same UI files

Bad Parallel Pairings

Avoid these combinations in the same active time window:

- both tools editing `src/PixelVault.Native/PixelVault.Native.cs`
- both tools editing `src/PixelVault.Native/Import/ImportWorkflow.cs`
- one tool changing service interfaces while the other tool simultaneously rewires callers to an older version of that interface
- one tool renaming/moving files while the other is patching those same files

Suggested Next Steps

If the goal is productive parallel work with low merge friction, the strongest next move is:

1. Cursor owns Library/MainWindow UI extraction.
2. Codex owns `ISettingsService` extraction in new files plus tests.
3. After that lands, Codex can take `IImportService` extraction while Cursor continues Library host work.
4. Do integration touches in `MainWindow` one slice at a time, with one owner per integration commit.

That sequence keeps the highest-conflict file (`PixelVault.Native.cs`) under one owner most of the time while still letting the service layer move forward.

Quick Prompt Templates

Prompt for Cursor

Continue the MainWindow split by extracting Library UI composition out of `PixelVault.Native.cs` into dedicated UI host files under `src/PixelVault.Native/UI/Library`. Preserve behavior. Avoid changing service interfaces unless necessary.

Prompt for Codex

Extract the next low-risk service from MainWindow in small safe steps. Start with `ISettingsService` in `src/PixelVault.Native/Services/Config`, keep behavior unchanged, add focused tests, and avoid touching Library UI files unless needed for final wiring.

Definition Of A Good Slice

A slice is ready to merge when:

- it has a clear owner
- it mostly touches one area of the tree
- it compiles
- it has at least one focused regression check
- it reduces what `MainWindow` directly knows about